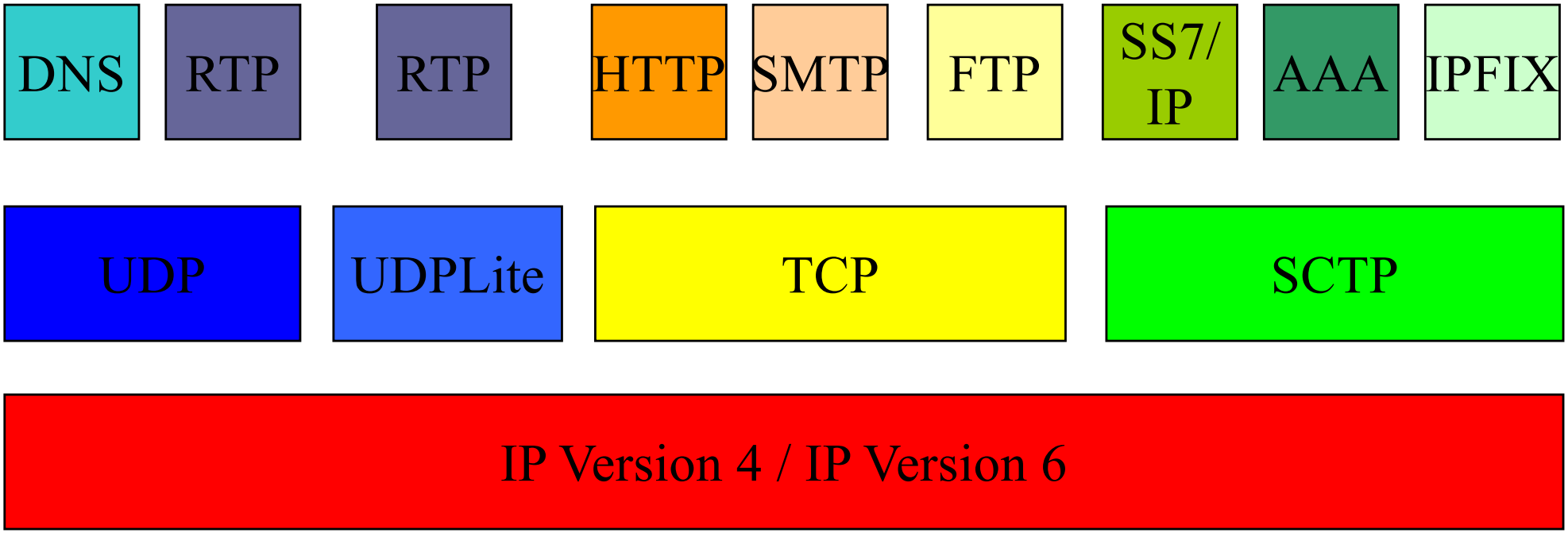
Netzwerkprogrammierung

Vorlesung 2

Die Internet Protocol (IP) Protokollfamilie



socket call

- int socket(int family, int type, int protocol);
  - family: AF_INET, AF_INET6, ...
  - type: SOCK_STREAM, SOCK_DGRAM, ...
  - protocol: 0, IPPROTO_UDP, IPPROTO_TCP, ...
- Gibt einen Filedescriptor >= 0 zurück oder im Fehlerfall -1.

bind call

- int bind(int fd, const struct sockaddr *addr,
  socklen_t addrlen);
  - fd: Filedescriptor
  - addr: Zeiger auf lokale Transportadresse
  - addrlen: Länge der lokalen Transportadresse
- Gibt 0 zurück, falls alles geklappt hat, -1 im Fehlerfall.

recvfrom call

- ssize_t recvfrom(int fd, void *buf, size_t buflen,
  int flags, struct sockaddr *from,
  socklen_t *addrlen)
  - fd: Filedescriptor
  - buf: Zeiger auf Speicherbereich
  - buflen: Länger des Speicherbereiches
  - from: Zeiger auf Ursprungstransportadresse
  - addrlen: Zeiger auf Länge der Ursprungstransportadresse (Value-Result argument)
- Gibt die Anzahl der gelesenen Bytes zurück, -1 im Fehlerfall.

sendto call

- ssize_t sendto(int fd, void *buf, size_t buflen,
  int flags, const struct sockaddr *to,
  socklen_t addrlen)
  - fd: Filedescriptor
  - buf: Zeiger auf Speicherbereich
  - buflen: Länger des Speicherbereiches
  - to: Zeiger auf Zieltransportadresse
  - addrlen: Länge der Zieltransportadresse
- Gibt die Anzahl der an den Kernel übergebenen Bytes zurück, -1 im Fehlerfall.

close call

- int close(int fd);
  - fd: Filedescriptor
- Gibt 0 bei Erfolg zurück, -1 im Fehlerfall.

Strukturen

- ```
  struct sockaddr {
    u_char  sa_len;              /* total length */
    u_char  sa_family;           /* address family */
    char    sa_data[14];         /* actually longer; address value*/
  };                             /* from sys/socket.h */
  ```
- ```
  struct sockaddr_in {
    u_char  sin_len;
    u_char  sin_family;
    u_short sin_port;
    struct  in_addr sin_addr;
    char    sin_zero[8];
  };                             /* from netinet/in.h */
  ```
- ```
  struct in_addr {
    in_addr_t s_addr;
  };                             /* from netinet/in.h */
  ```
  ```

Network Byte Order

- int16 Werte brauchen 2 Bytes im Speicher.
- Ein High Order Byte, ein Low Order Byte.
- Litte Endian: Low Order Byte steht an niedrigerer Adresse.
- Big Endian: Low Order Byte steht an höherer Adresse.
- Big Endian = Network Byte Order.

Byte Order Conversion

- uint16_t htons(uint16_t);
- uint16_t ntohs(uint16_t);
- uint32_t htonl(uint32_t);
- uint32_t ntohl(uint32_t);

Echo Server

```
int main()
{
    int fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len;
    int len;
    char buf[1<<16];
    fd  = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family      = AF_INET;
    server_addr.sin_len         = sizeof(struct sockaddr_in);
    server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
    server_addr.sin_port        = htons(7);
    bind(fd, (const struct sockaddr *) &server_addr, sizeof(server_addr));

    while (1) {
        addr_len = sizeof(client_addr);
        memset(&client_addr, 0, sizeof(client_addr));
        len      = recvfrom(fd, (void *) buf, sizeof(buf), 0,
                            (struct sockaddr *) &client_addr, &addr_len);
        sendto(fd, (const void *) buf, len, 0,
               (const struct sockaddr *) &client_addr, addr_len);
    }
}
```

Elementare Fehlerbehandlung

- Ersetze socket(fd, type, protocol) durch
- int Socket(int fd, int type, int protocol)

```
{
        int n;
        if ((n=socket(fd, type, protocol)) < 0) {
                perror(„socket“);
                exit(-1);
        }
        return(n);
}
```

Internet Control Message Protocol

- Kein ‚Transportprotokoll‘.
- Upper Layer von IPv4.
- Viele verschiedene Typen:
  - Echo Request
  - Echo Reply
  - Destination Unreachable
- `ping` benutzt ICMP.

ICMP und UDP Sockets

- ICMP dient zur Fehlerbehandlung bei UDP.
- Wann wird was gesendet?
- Wie wird dies im API signalisiert?

connect call

- int connect(int fd,
  const struct sockaddr *addr,
  socklen_t len)
  - fd: Filedescriptor;
  - addr: Zeiger auf Adresse des Peers;
  - len: Länger der Adresse.
- Gibt 0 bei Erfolg zurück, -1 im Fehlerfall.

recv call

- ssize_t recv(int fd, void *buf, size_t buflen, int flags)
  - fd: Filedescriptor
  - buf: Zeiger auf Speicherbereich
  - buflen: Länger des Speicherbereiches
- Gibt die Anzahl der gelesenen Bytes zurück, -1 im Fehlerfall.

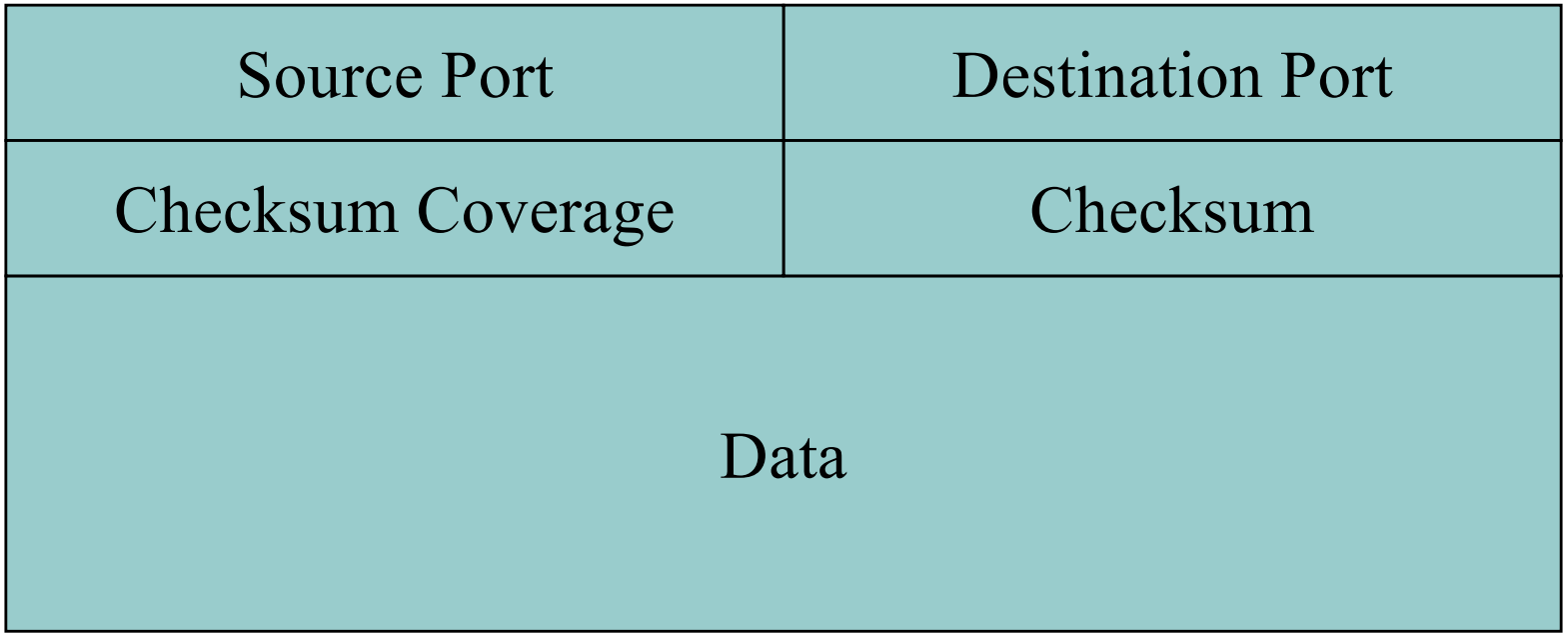
send call

- ssize_t send (int fd, void *buf, size_t buflen, int flags)
  - fd: Filedescriptor
  - buf: Zeiger auf Speicherbereich
  - buflen: Länger des Speicherbereiches
- Gibt die Anzahl der gesendeten Bytes zurück, -1 im Fehlerfall.

setsockopt/getsockopt call

- int setsockopt(int fd, int level, int optname, const void *optval, socklen_t optlen)
  - fd: Filedescriptor.
  - level: SOL_SOCKET, IPPROTO_IP, IPPROTO_TCP, ...
  - optname: SO_REUSEADDR, IP_HDRINCL, IP_TTL, ...
  - optval, optlen: Wert, der gesetzt wird.
- int getsockopt(int fd, int level, int optname, void *optval, socklen_t *optlen)

UDPLite Packet



Checksum Coverage

- Checksum Coverage
  - 0: Komplette Paket wie bei UDP.
  - 1-7: Nicht erlaubt.
  - 8-Paketlänge: Nur die angegebenen Bytes werden berücksichtigt.
  - Größer als Paketlänge: Nicht erlaubt
- Socket Options
  - UDPLITE_SEND_CSCOV
  - UDPLITE_RECV_CSCOV

Probleme bei der Programmierung

- socklen_t nicht verfügbar.
- sin_len nicht immer vorhanden.
- Anzahl der Filedescriptoren sehr unterschiedlich.
- Unterschiedliche Headerfiles.
- Unterschiedliche Bibliotheken.

Handhabung von Typen

- Das configure Script testet, ob die entsprechenden Typen vorhanden sind. Falls nicht, werden Ersatztypen definiert.
- AC_CHECK_TYPE(ssize_t)
- AC_CHECK_TYPE(socklen_t) funktioniert nicht, da nicht in sys/types.h definiert

Handhabung von Strukturkomponenten

- Hierzu compiliert man ein Testprogramm. Wenn es eine benutzte Komponente nicht gibt, erzeugt der Compiler einen Fehler.
- AC_TRY_COMPILE(incl, body, ok, else)
  - Es wird versucht, dass Programm mit den includes incl und dem Body body zu übersetzen. Im Erfolgsfall werden die Kommandos ok ausgeführt, sonst else.
- AC_DEFINE(identifier)
  - identifier wird definiert
- Anwendung: siehe Beispiel

Handhabung von Funktionen

- AC_CHECK_FUNCS(func, ok, else)
  - ‚HAVE_FUNC' ist gegebenenfalls definiert, die ok Kommandos ausgeführt.
  - sonst else Kommandos ausgeführt.
- AC_CHECK_LIB(lib, func)
  - Prüft, ob func in lib definiert ist und fügt lib gegenenfalls zu den zu benutzenden Libraries hinzu.
- Sollte auch kombiniert werden, siehe Beispiel.

Prüfen der Umgebung

- AC_CANONICAL_HOST
- cpu-manufacturer-os triplet
  - Beispiel: powerpc-apple-darwin6.5
- case $host_os in
  solaris*)
  CFLAGS="$CFLAGS -DSOLARIS“
  ;;
  esac
- Im Code dann mit #ifdef SOLARIS arbeiten

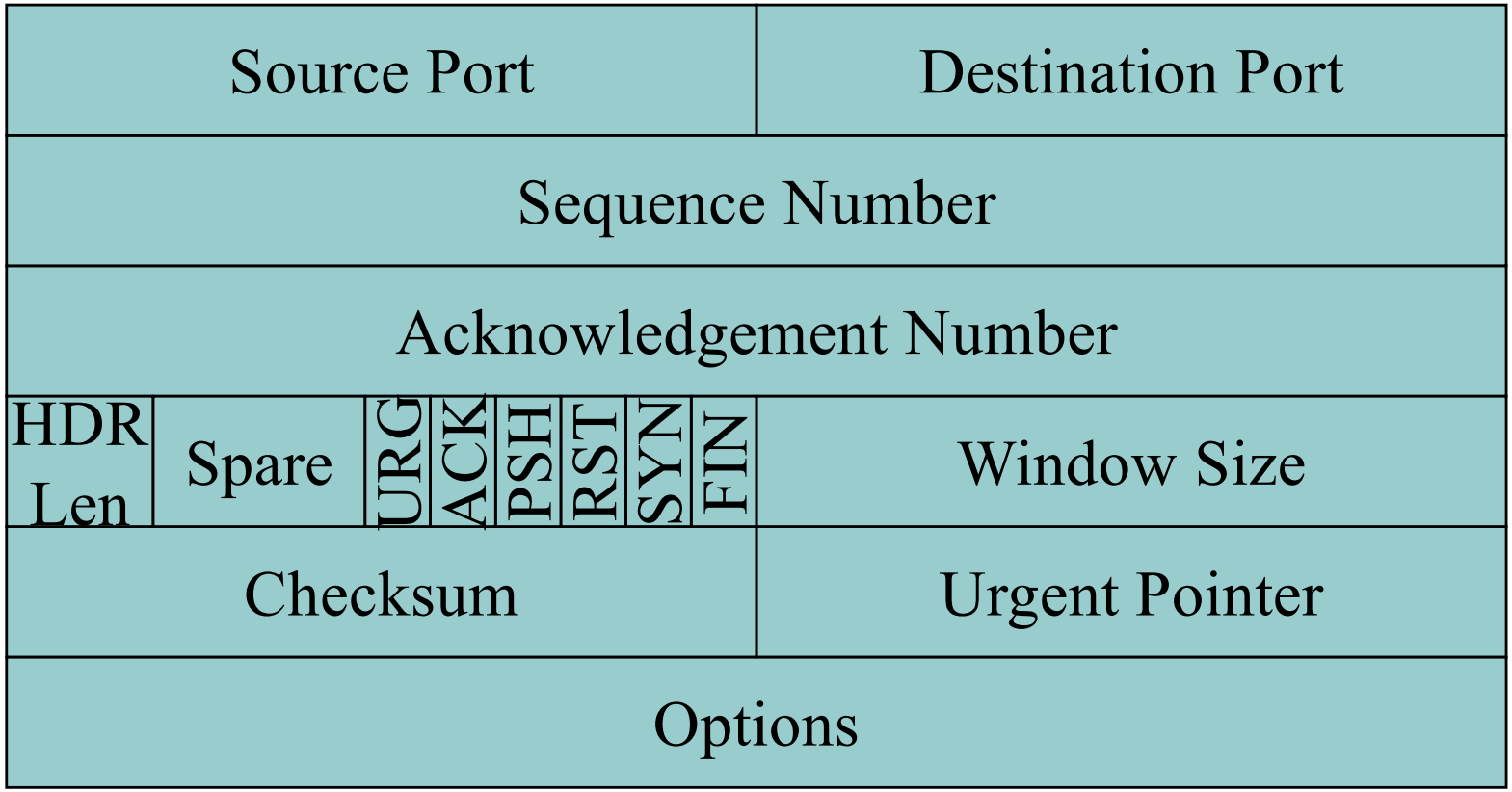
Beispiel: udp_examples

- Der Source wurde so modifiziert, dass er unter Linux, Solaris, FreeBSD und MacOSX läuft.
- Unterschied: Entwickler <-> User.
- Erstellt werden müssen bootstrap, Makefile.am und configure.in.

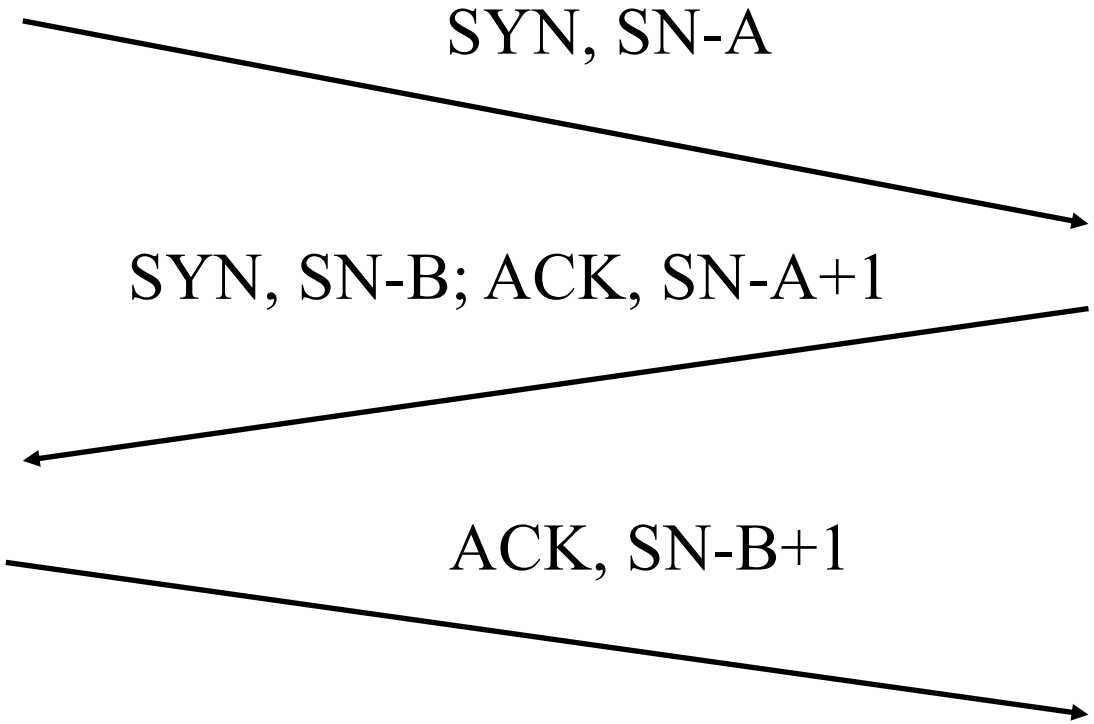
Transmission Control Protocol

- Bytestreamorientiert
- Gesichert
- Flowcontrol
- Congestioncontrol
- Segmentierung
- Singlehoming
- Unicast

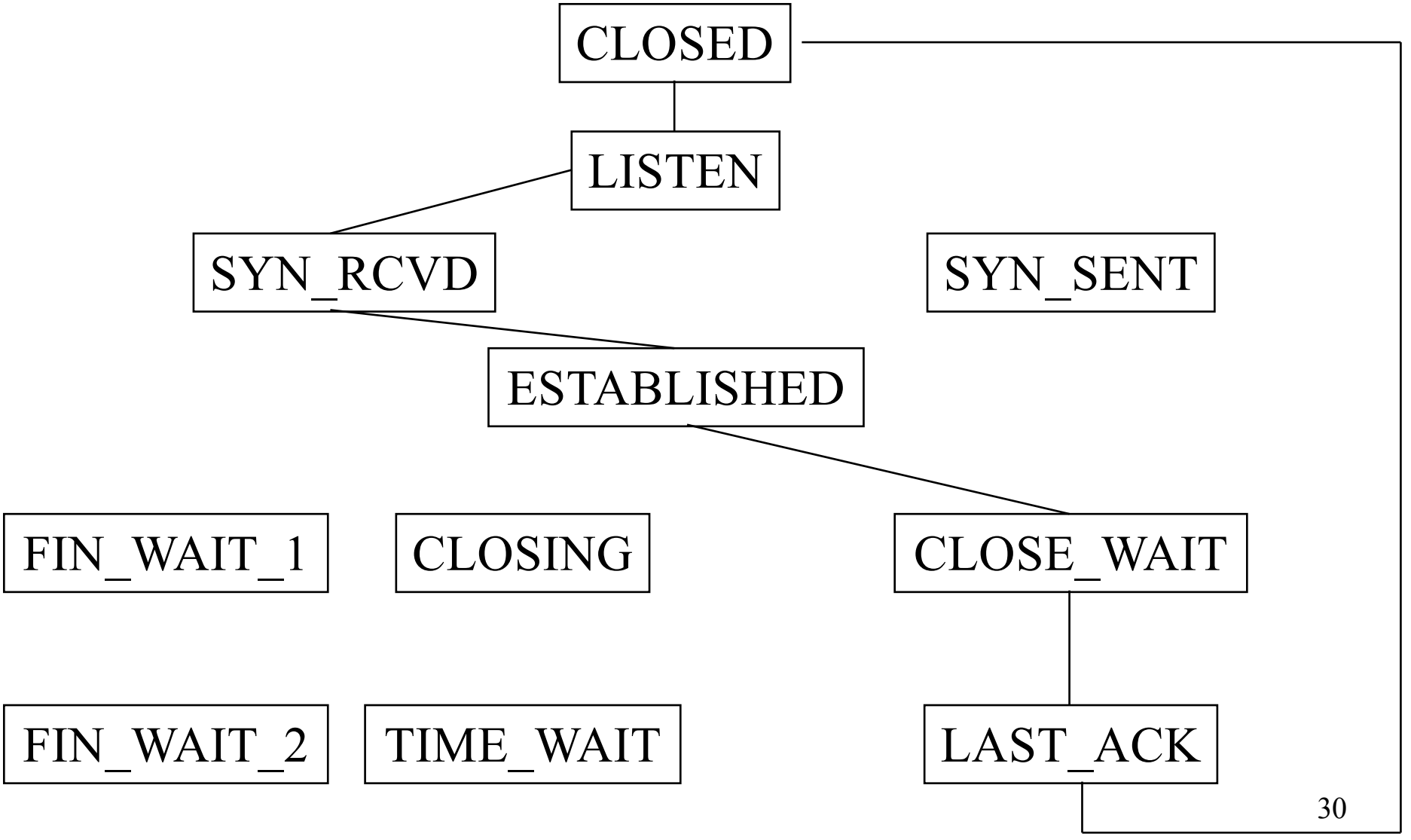
TCP Header


Source Port
Destination Port
Sequence Number
Acknowledgement Number
HDR Len
Spare
URG
ACK
PSH
RST
SYN
FIN
Window Size
Checksum
Urgent Pointer
Options

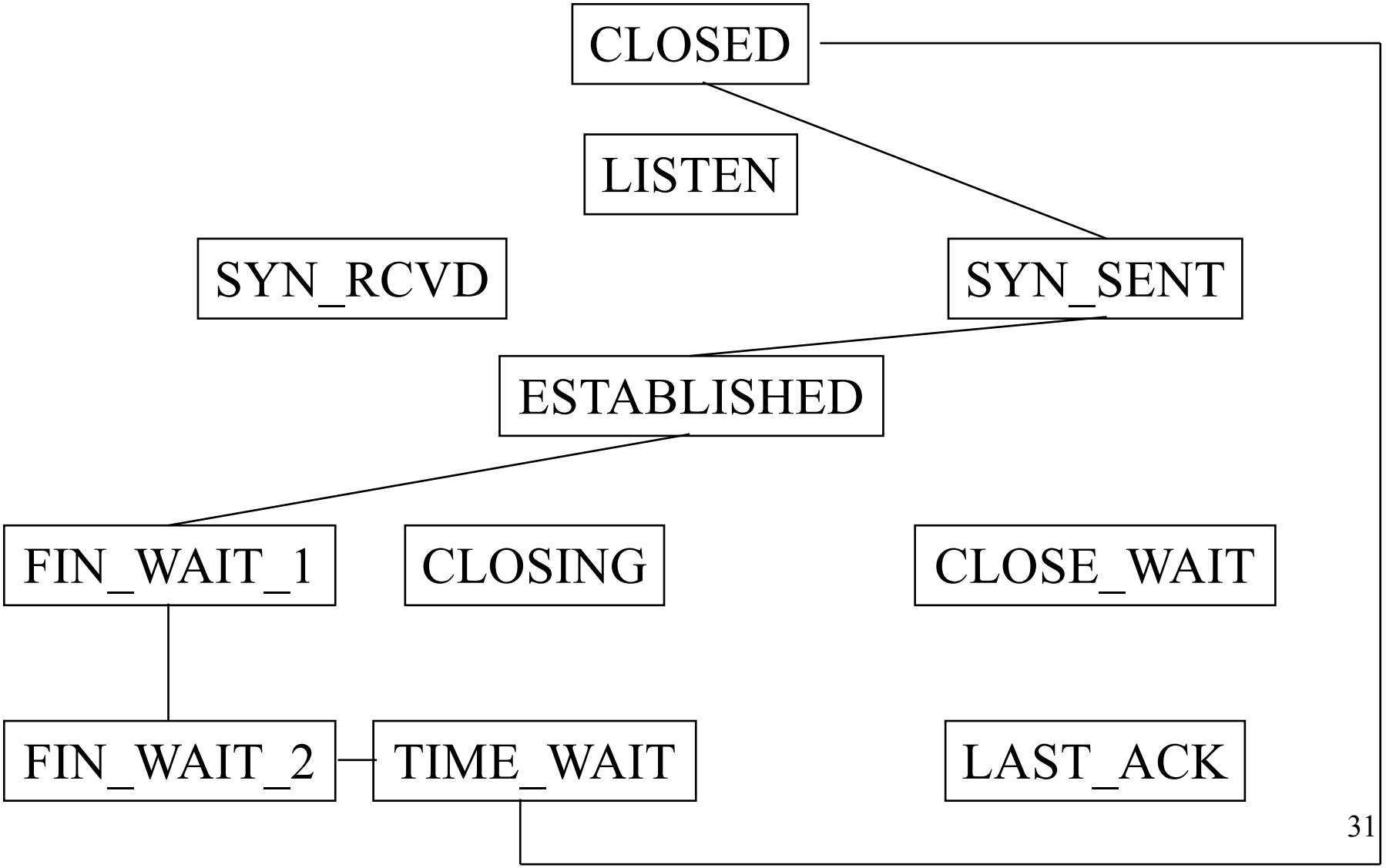
Der Verbindungsaufbau



TCP State Engine (Server)



TCP State Engine (Client)



listen call

- int listen(int fd, int backlog);
  - fd: Filedecriptor
  - backlog: Anzahl der maximal zu puffernden Verbindungsanforderungen.
- Gibt 0 zurück, wenn alles geklappt hat, sonst -1.

accept call

- int accept(int fd,
  struct sockaddr* addr, socklen_t *len)
  - fd: Filedescriptor.
  - addr: Pointer auf Addressstruktur
  - len: Pointer auf Länge. Value-Result Parameter
- Gibt einen Filedescriptor für die akzeptierte Verbindung zurück oder -1 im Fehlerfall.

connect call

- int connect(int fd,
  const struct sockaddr *addr,
  socklen_t len)
  - fd: Filedescriptor;
  - addr: Zeiger auf Adresse des Servers;
  - len: Länger der Adresse.
- Gibt 0 bei Erfolg zurück, -1 im Fehlerfall.

close call

- int close(int fd);
  - fd: Filedescriptor
- Gibt 0 bei Erfolg zurück, -1 im Fehlerfall.

shutdown call

- int shutdown(int fd, int how);
- how ist SHUT_RD, SHUT_WR, SHUT_RDWR.
- Gibt 0 bei Erfolg zurück, -1 im Fehlerfall.

Discard Server

- Well known Port: 9.
  - socket()
  - bind()
  - listen()
  - accept()
  - recv()
  - close()

Ein Discard Server (vereinfacht)

```
fd  = Socket(AF_INET, SOCK_STREAM, 0)
memset((void *) &server_addr, 0, sizeof(server_addr));
server_addr.sin_family      = AF_INET;
server_addr.sin_len         = sizeof(struct sockaddr_in);
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
server_addr.sin_port        = htons(DISCARD_PORT);
Bind(fd, (const struct sockaddr *) &server_addr, sizeof(server_addr))
Listen(fd, 1);
while (1) {
        memset((void *) &client_addr, 0, sizeof(client_addr));
        client_addr_len = (socklen_t) sizeof(client_addr);
        cfd = Accept(fd, (struct sockaddr *) &client_addr,
                         &client_addr_len)
        do {
                len = Recv(cfd, (void *) buf, sizeof(buf), 0);
        } while (len > 0)
        Close(cfd);
}
Close(fd);
return(0);
```

Problem beim Server

- Bricht man einen Server mit Control-C ab, wenn er noch eine Verbindung mit einem Client hat, so kann man ihn nicht sofort wieder starten.
- Der socket befindet sich im State TimeWait.
- Der bind Aufruf gibt einen Fehler zurück.